LightRIM: Light Runtime Integrity Measurement for Linux Kernels in Embedded Applications

Yili Guo[1], Zhuoran Ma[1], Xiangyue Li[1], Jiajia Huang[1], Wanli Chang[1,2]
[1]Hunan University, China [2]Huawei Technologies, China

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132602

***Abstract*—Linux kernels are being widely deployed in embedded applications, such as increasingly automated vehicles and robots, due to their robust ecosystem. Security modules have been developed to enhance the integrity of Linux kernels, a critical system component. However, these modules consume substantial computational resources, making them unsuitable for embedded domains. We introduce LightRIM, a lightweight method to measure the Linux kernel's integrity during runtime, ideal for resource-limited embedded applications. We focus on major attack types and extract objects for monitoring. Our approach includes a two-stage hashing process and an event-triggered measurement algorithm tied to the security value. To mitigate Time-of-Check-to-Time-of-Use (TOCTOU) attacks, we introduce a heuristic algorithm that maximizes the attack detection rate within CPU usage constraint and randomizes the measurement intervals. Experimental results indicate that LightRIM incurs less than 0.7% performance overhead while providing extensive attack coverage.**

I. Introduction

Linux kernels are prevalent in embedded systems, particularly in intelligent and connected applications, owing to their strong ecosystem and dependability. Security is of high importance, as any breach of kernel integrity can lead to dire outcomes. Unauthorized alterations to kernel code or data structures may cause system malfunctions, vulnerabilities, or total collapse. Thus, preserving the integrity of Linux kernels during runtime is vital for the security and stability of the entire embedded system.

Linux Security Module (LSM) has been developed to enhance the integrity of the Linux kernel. Among its components, the Integrity Measurement Architecture (IMA), introduced in 2004 and based on Trusted Computing Group (TCG) standards, has driven significant advancements in both static and dynamic integrity measurement techniques [1], [2]. Nevertheless, IMA's hash algorithms incur significant computational overheads [3], and measuring large files can result in substantial storage demands [4].

Related Works: Jaeger et al. [5] proposed PRIMA, a prototype based on IMA and integrated with SELinux policies to focus on trusted subject interactions, reducing overhead. However, its reliance on PRIMA-aware applications and the required OS modifications limit its practicality. Zhang et al. [6] demonstrated that SELinux integration significantly degrades kernel performance, even with policy enforcement disabled. Then, DRIVE [7] introduced a method for detecting runtime attacks by continuously monitoring memory images of binary files and verifying their consistency with the loaded binary code. However, this approach mainly focuses on memory image measurements, overlooking other critical objects, which limits its comprehensiveness. Meanwhile, Xfilter [8] provides an LSM policy framework with package management integration but relies on extensive customization and system-wide monitoring, making it less suitable for embedded systems. Bohling et al. [9] demonstrated that IMA's security guarantees can be undermined by malicious block devices and investigated inherent Time-of-Check-to-Time-of-Use (TOCTOU) vulnerabilities within Linux's IMA framework. The problem of TOCTOU invalidity is widespread in various fields such as remote attestation, operating systems, and certifications. Numerous approaches have emerged to address TOCTOU vulnerabilities across diverse contexts [10], [11]. However, these methods mainly rely on hardware-based remote attestation to counteract TOCTOU attacks. Embedded systems frequently depend on hardware-supported secure boot mechanisms to ensure system integrity at the hardware level. These mechanisms include secure boot chains, roots of trust, and bootloaders [12]. Although these solutions provide robust security during booting, they cannot prevent run-time manipulation and often impose significant hardware resource overhead [13].

This paper presents LightRIM, a lightweight runtime integrity measurement architecture for monitoring immutable variables on Linux systems. Critical measurement objects, identified through integrity-compromising attack analysis, are hashed and stored in a baseline library. To minimize storage, a two-stage hashing mechanism aggregates these objects into a single hash. An event-triggered measurement algorithm dynamically adjusts security values for finer granularity. To mitigate TOCTOU attacks, we model the relationship between system overhead and measurement intervals, proposing an optimization algorithm to maximize detection rates under CPU constraints. Benchmarks and real-world attack evaluations validate LightRIM's efficiency and robustness.

The contributions of this paper are as follows:

- We develop a baseline library for storing critical runtime objects and propose a two-stage hashing mechanism with an event-triggered measurement algorithm.
- We formulate a Simulated Annealing (SA)-based optimization algorithm to maximize TOCTOU attack detection rates under predefined CPU usage constraints.
- We demonstrate that LightRIM incurs less than 1 ms latency in LMBench tests and increases system overhead by

This work was supported by the National Key Research and Development Program of China (2023YFB4503704).

Corresponding Author: Wanli Chang (wanli.chang.rts@gmail.com).

under 0.7% in SPEC CPU2006 tests, and exhibits robust detection capabilities against real-world attacks. Our code is available at https://github.com/ElyerGuo/LightRIM.git.

II. Background

A. Linux Kernel Integrity Attacks

The ATT&CK framework is a globally accessible knowledge base constructed on real-world attack vectors. From this framework, we select Linux integrity-related attacks, focusing on two representative examples: code injection and rootkit attacks.

Code Injection Attack. The ATT&CK library includes attacks such as Virtual Dynamic Shared Object (VDSO) Hijacking (T1055.014), Ptrace System Calls (T1055.008), and Dynamic Linker Hijacking (T1574.006). Taking VDSO as an example, it is a dynamic library in the kernel that can be mapped to the user space. Adversaries may hijack the system call interface code stubs mapped from VDSO shared objects to processes to execute system calls and open and map malicious shared objects. This attack involves modifying the kernel VDSO page with the set_memory_rw function, allowing user space to directly modify VDSO and hijack it for shellcode (kernel arbitrary code execution vulnerability). Next, it modifies the kernel VDSO data and writes shellcode to hijack functions in the user space. When a high-privileged process calls specific functions in VDSO, it triggers the shellcode (arbitrary kernel read/write vulnerability).

Rootkit Attack. The ATT&CK library provides procedural examples for various rootkit attacks (T1014). Adversaries usually manipulate the core components or system tools of the operating system using rootkit attacks, allowing them to gain unauthorized access to the system and maintain long-term control. For example, adversaries may modify the source code of the Linux kernel or load malicious kernel modules to take control and conceal the system. Additionally, they may alter the behavior of system tools such as `ps` and `ls` to conceal their activities or provide false information.

B. Threat Model

As a runtime integrity measurement solution for Linux systems, our threat model and assumptions align with prior research [14]–[16]. The primary considerations are as follows:

- We assume that the Linux system is correctly loaded, fully functional at boot, and actively running, though it may contain vulnerabilities that allow attackers to interact with the kernel post-boot.
- The attacker possesses complete control of non-root user-mode processes and can interact with the kernel through standard kernel interfaces.
- Other kernel integrity measurement methods, such as IMA and control flow integrity (CFI) techniques, are assumed to be established.
- LightRIM is assumed to be deployed in a trusted setting, like a Trusted Platform Module (TPM)-protected environment, safeguarding the integrity of the baseline library against unauthorized modifications.

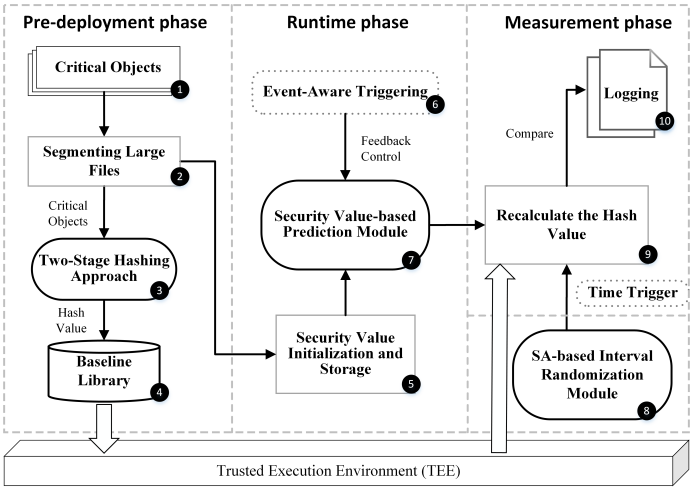


Fig. 1: Overview of LightRIM.

In this threat model, the attacker's objective is to tamper with critical system components to compromise the kernel, escalate privileges, or disrupt normal system operation.

III. Lightweight Runtime Integrity Measurement Framework Design

A. LightRIM Framework Overview

As shown in Fig. 1, the lightweight runtime integrity measurement framework consists of three phases: the Pre-deployment, Runtime, and Measurement phases. In the Pre-deployment phase, ❶ all critical objects defined by integrity attack models and immutable during runtime are extracted from the OS kernel and physical memory. ❷ Large objects, such as the kernel code and data segments, are divided into smaller sections to reduce system load. ❸ A two-stage hashing approach generates hash values for these objects, which are then ❹ stored in the baseline library. During the Runtime phase, ❺ critical object's security values are initialized and stored in the prediction module. ❻ An event-triggered mechanism evaluates measurement conditions, while ❼ security values are dynamically updated via the kernel notification chain based on system events. In the Measurement phase, ❽ an SA-based interval randomization schedules measurement operations to counter TOCTOU attacks. ❾ The current hash values of critical objects are recomputed and compared with the baseline. Any mismatch ❿ triggers an alert, signaling a possible integrity breach. This design ensures efficient integrity monitoring with minimized overhead and enhanced security.

B. Identifying and Extracting Critical Objects

In various types of integrity attacks, distinct attack targets have critical measurement objects that are vulnerable and require regular protection. Code injection attacks commonly target areas such as the kernel code segment, kernel data segment, and kernel libraries, attempting to alter the kernel's execution flow. Rootkit attacks, on the other hand, are more likely to compromise critical data structures like the system call table and interrupt descriptor table, aiming to conceal

malicious actions or establish persistent control. Additionally, user-space system processes are frequently targeted by malicious threats, further elevating the overall risk to the system. These attacks often exploit various techniques to infiltrate the system during runtime, thereby compromising the operating system's integrity. Consequently, ensuring the real-time protection of these critical objects is essential to maintain system security.

To protect critical information such as the kernel code segment, kernel data segment, and system call table, we employ the `kallsyms_lookup_name` kernel function. This function allows us to locate the addresses of kernel symbols (typically functions or variable names) within the kernel space by their symbol names. Using these addresses, we can access and extract the contents of the corresponding measurement objects for integrity protection. For kernel libraries, such as the VDSO, we directly secure the user-space VDSO library. Regarding process-related information, we access the `mm_struct` to obtain the process address space and apply protection to critical information at the corresponding addresses. Our approach specifically targets processes present at system startup, excluding any processes initiated during runtime.

C. Coarse-Grained Two-Stage Hashing Approach

This subsection describes a two-stage hashing mechanism designed to reduce the storage and comparison overhead of the baseline library. As shown in Fig. 2, we first evaluated several widely used hash algorithms within the Linux system to identify a suitable trade-off between performance and security. Among these, SHA-256 was selected due to its robustness and acceptable computational cost.

In the first stage, we apply the SHA-256 algorithm to each of the extracted measurement objects, which include kernel code and data segments, system call tables, the file system, kernel modules, and user process memory regions. For each object, SHA-256 computes a 256-bit hash value that uniquely represents its content. These individual hash values are stored in the baseline library and denoted as $HV_1, HV_2, \ldots$ in Fig. 2. To further reduce the verification overhead during runtime, a second-stage aggregation process is introduced. Specifically, only the first 32 bits of each individual hash value are selected and computed into a intermediate vector. The vector is then re-hashed using SHA-256 to generate a final aggregated hash value, referred to as `ALL_HASH_VALUE`. Both the individual hash values and this final aggregated hash value are stored in the reference database using kernel file operation functions. This two-stage approach enables fast integrity verification with minimal storage and computational costs.

D. Fine-Grained Event-Triggered Measurement Algorithm

This subsection will cover the fine-grained event-triggered measurement algorithm, consisting of the security value-based prediction and the event-aware modules.

Security Value-Based Prediction Module. The prediction module leverages dynamically adjusted security values to determine whether a measurement object should be evaluated during each cycle. At the start of measurement, a random value is generated using a random seed and compared against the object's current security value. If the security value is lower, the object is measured, and its security value increases; otherwise, the security value decreases, raising its likelihood of measurement in subsequent cycles. Upon detecting an attack, the object's security value is significantly reduced, substantially increasing its measurement probability in future cycles. Additionally, an integrated event-triggering module dynamically adjusts security values based on system events, ensuring adaptability to runtime conditions.

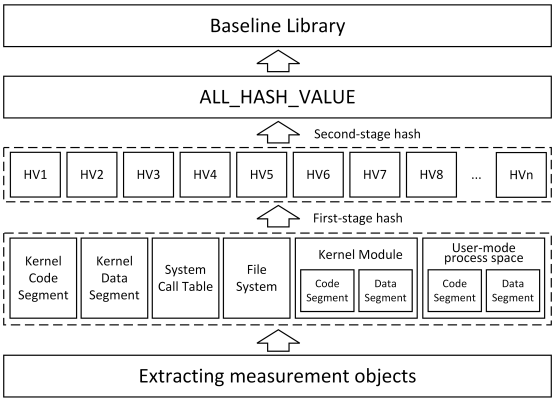


Fig. 2: The architecture of the two-stage hashing process.

Event-Aware Triggering. Deploying our architecture in kernel mode requires real-time detection of critical user-space events, including process creation/termination, kernel module loading/unloading, changes to process code segment permissions, and shell script execution. These behaviors indicate potential changes to the system's integrity in both kernel and user spaces. By monitoring these events, the architecture substantially improves overall system security. Since all these behaviors depend on system calls bridging user and kernel spaces, they can be effectively tracked by instrumenting system call functions. Algorithm 1 outlines the event-triggering mechanism that informs the measurement module and initiates necessary actions. Initially, a kernel notification chain is established, enabling kernel components to register as observers for specific events (line 2). The measurement module registers a notification function to handle event updates (lines 3–5). When an instrumented function detects a critical system call, it triggers the notification chain and passes a notification number to the measurement module (line 6). The measurement module then processes the event by adjusting security values as required (lines 7–10).

E. SA-based Attack Detection Rate Optimization Algorithm

The periodic measurement approach is vulnerable to TOCTOU attacks, as attackers can predict and exploit fixed intervals. Current solutions overlook the balance between randomized measurement intervals and system overhead, and lack a systematic model for attack detection rates. Frequent measurement triggers can lead to significant system overhead,

Algorithm 1 Event Aware Module

Input: Event i is triggered
1: $S_i \leftarrow$ The security value of object i changed
2: *RAW_NOTIFIER_HEAD(notifier_chain)* ▷ Define kernel notification chain header
3: *kprobe_trigger_function* ▷ Kprobe definition
4: *orig ← kallsyms_lookup_name(trigger_function)* ▷ Find the function address
5: *register_kprobe(&kprobe_trigger_function)* ▷ Register kprobe
6: *handler_pre_trigger ← notify_observers(i)*
7: The kernel notification, indexed by i, precedes the instrumentation function trigger
8: **if** *notifier_callback* == i **then**
9: ADD(S_i) ▷ Change the security value
10: **end if**

Algorithm 2 SA-based Attack Detection Rate Optimization Algorithm

Input: ω, $u_{\max}$
Output: θ, $\lambda_{\max}$
1: T_{ini}=100, T_{ter}=0.01, δ= 0.98;
2: $m_{\text{k}} \leftarrow$ Compute from $u_{\max}$ using Equation (1);
3: $\theta_{\text{k}} \leftarrow$ `Generate_Time_Points`$(m_{\text{k}}, n \times m_{\text{k}})$;
4: $\lambda_{\text{k}} \leftarrow$ `Simulate_TOCTOU_Attack`$(\omega, \theta_{\text{k}})$;
5: $T = T_{\text{ini}}$, $\lambda_{\max} = \lambda_{\text{cur}} = \lambda_{\text{k}}$;
6: **while** $T > T_{\text{ter}}$ **do**
7: $\theta_{k'} \leftarrow$ `Generate_Time_Points`(m_i, m_j) ▷ $m_i, m_j \in (m_k, n \times m_k)$
8: $\lambda_{k'} \leftarrow$ `Simulate_TOCTOU_Attack`$(\omega, \theta_{k'})$
9: **if** $\lambda_{k'} > \lambda_{\text{k}}$ **or** $e^{(\lambda_{k'} - \lambda_{\text{k}}) \div T} > random(0,1)$ **then**
10: $\theta_{\text{k}} = \theta_{k'}$, $\lambda_{\text{cur}} = \lambda_{k'}$;
11: **if** $\lambda_{\max} < \lambda_{k'}$ **then**
12: $\lambda_{\max} = \lambda_{k'}$, $\theta = \theta_{k'}$;
13: Save the randomly generated range (m_i, m_j);
14: **end if**
15: **end if**
16: $T = T \times \delta$
17: **end while**

while infrequent measurements drastically reduce attack detection rates. Therefore, it is essential to design a randomized measurement time sequence that balances these trade-offs, maximizing attack detection rates while satisfying CPU usage constraints in limited systems. To address these problems, we developed a model that quantifies system overhead in terms of CPU usage as a function of measurement intervals and designed an SA-based heuristic algorithm to maximize detection rates within predefined CPU usage constraints, thereby reducing TOCTOU vulnerabilities.

1) Model Fit Function: To quantitatively assess the impact of measurement interval lengths on CPU usage, we conducted over 100 trials per interval on the experiment platform described in Section IV, without additional user-initiated processes. The measurement intervals ranged from 0 to 10,000 milliseconds, and average CPU usage was determined for each interval. Given the observed oscillations and exponential decay in CPU usage, we applied a logarithmic transformation to linearize the data, which followed the exponential function $y = ae^{-bx} + c$. This suggested an exponential decay model, with variations expected across different platforms. Subsequently, we employed Python's function $scipy.optimize.curve\_fit$, leveraging the Levenberg-Marquardt algorithm for nonlinear least squares fitting, to fit this model to our data. The resulting fitted equation is as follows:

$$\ln(u_i) = 2.64458 \times e^{-0.00023 \times m_i} - 0.3952 \qquad (1)$$

where u_i represents the average CPU usage when the measurement module is active and m_i denotes the specified periodic measurement interval. From our calculations, the fitted function in Equation 1 achieved a coefficient of determination (R^2) of 0.99 and a Root Mean Square Error (RMSE) of 0.25, demonstrating a highly effective fit.

2) SA-based Optimization Algorithm Design: The SA-based heuristic algorithm is widely recognized for efficiently identifying optimized solutions within a search space. We propose an SA-based algorithm to generate randomized measurement time points , enhancing the detection rate of TOCTOU attacks. The algorithm is detailed in Algorithm 2, takes the vulnerability time window ω and CPU usage upper limit $u_{\max}$ as inputs, and output the optimized measurement sequence θ along with the maximum TOCTOU attack detection rate $\lambda_{\max}$. The algorithm first initializes the SA parameters (lines 1-5). Then it generates the minimal measurement interval m_{k} under the constraint $u_{\max}$, and an initial measurement sequence θ_{k} using the `Generate_Time_Points` function. The initial TOCTOU detection rate is computed using the `Simulate_TOCTOU_Attack` function. We simulate discrete attack events randomly generated within a time window ω, where a modification is applied to a specific measurement object. Assuming that the attacker requires a certain amount of time to execute the operation, the attack is considered detected if the measurement point identifies the event. The attack detection rate λ is defined as the ratio of detected attacks to the total number of attacks. Then, the SA temperature iteration is performed (lines 6-17). To generate a new time sequence, the `Generate_Time_Points` function moves within the solution space from m_{k} to $n \times m_{\text{k}}$, generating a new random interval (m_i, m_j). After generating the sequence, an attack simulation yields the detection rate $\lambda_{k'}$. The new sequence is accepted if it shows a higher detection rate or meets the probability acceptance criterion (lines 9-15). If $\lambda_{k'} > \lambda_{max}$, the θ, λ_{max} are updated (lines 11-13).

IV. Evaluation

A. Implementation

LightRIM is designed to be easily integrated into the Linux kernel with root privileges, enabling straightforward deployment across diverse Linux systems. This approach significantly streamlines the deployment process in practical applications.

Baseline Library Management. Dynamic measurement objects are recalculated during each reboot to reflect state changes, while static objects are updated only after system modifications.

Adaptive Measurement Strategy. An adaptive module dynamically switches between coarse-grained and fine-grained methods based on CPU load. Coarse-grained methods reduce overhead during high system loads, while fine-grained meth-

TABLE I: Comparison of Hashing Mechanism Overhead.

Category	CPU Usage (%)	Above Baseline(%)
Baseline	0.25	-
Hashing	2.056	1.806
Two-stage hashing	1.628	1.378

ods provide stronger security assurances under lighter loads, ensuring a robust yet efficient monitoring system.

B. Experimental Setup

1) Platform: The experiments were conducted on a 64-bit Ubuntu 22.04 system running Linux kernel version 5.10.0-1057-oem, featuring an 8-core Intel i5-12400 processor and 8GB RAM.

2) Benchmarks: We evaluate the runtime performance of LightRIM by applying two benchmarks: LMBench 3.0 for detailed component analysis and the SPEC CPU2006 suite for comprehensive system evaluation, as done in previous research [15], [17]. The benchmarks are first run on a system with IMA enabled to establish baseline performance. Next, our module is loaded into the Linux kernel with the measurement functionality enabled, followed by another round of benchmark tests. The results are labeled as IMA and LightRIM+IMA, respectively.

C. Comparison of CPU Usage between Hashing and Two-Stage Hashing

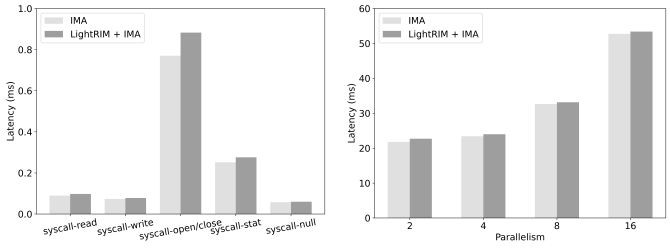
To evaluate the effectiveness of the coarse-grained measurement mechanism, we analyzed CPU usage under three scenarios: system idle, one-stage hashing, and two-stage hashing, as illustrated in Table I. The results demonstrate that the two-stage hashing mechanism reduces CPU overhead by 23.7% compared to one-stage hashing, excluding baseline overhead. This improvement is attributed to the design of the two-stage mechanism, which computes only `ALL_HASH_VALUE` during measurements. In contrast, one-stage hashing requires the extraction and comparison of multiple individual hash values, resulting in higher I/O and computational overhead.

D. Effectiveness of Attack Detection Rate Optimization

This subsection evaluates the performance of Algorithm 2 under CPU usage constraints ranging from 1% to 9%. For each constraint, 100 experiments were conducted, and average detection rates are presented in Table II. The *Initial* row shows the detection rate before optimization, while the *Optimal* row indicates the rates after optimization. The improvement (Δ) highlights the algorithm's impact, with the highest gain observed under stricter CPU constraints (e.g., a 43.6% increase at 1% CPU). However, as CPU availability rises, marginal improvement diminishes to 0.9% at 9%. Beyond this threshold, the difference between initial and optimized measurements become negligible, both approaching 100%. These results are attributed to the increased availability of system resources, which enables shorter measurement intervals, minimizing the temporal window for exploitation. However, practical systems rarely allocate excessive resources solely for runtime integrity monitoring, requiring a trade-off between security and performance. Thus, 9% is selected as a representative constraint for resource-scarce scenarios, and can be adjusted accordingly.

TABLE II: The average detection rate under CPU constraints.

Constraints	1%	2%	3%	4%	5%	6%	7%	8%	9%
Initial (%)	23.1	43.7	62.5	77.2	86.1	90.6	93.9	97.2	99.0
Optimal (%)	66.7	73.4	78.2	82.9	91.7	93.5	96.7	98.9	99.9
Δ (%)	43.6	29.7	15.7	5.7	5.6	2.9	2.8	1.7	0.9



(a) The comparison of syscall latency in LMBench tests.

(b) The comparison of context switch latency in LMBench tests.

Fig. 3: Performance overheads of LMBench.

Since all monitored objects are assumed static during runtime, any detected modifications are considered genuine compromises, resulting in a false positive rate of `0`. Conversely, true negatives represent undetected attacks, quantified as `1-detection rate`.

E. Performance Evaluation

1) LMBench: The experiments evaluated common system call operations, including read, write, and open/close, as well as context switch latency under parallel configurations of 2, 4, 8, and 16 processes. A standardized workload size of 1024 KB was used and each test was repeated 100 times to compute average latency values. To ensure consistency and minimize variability, a 10-second warm-up phase was applied before each test to stabilize system resources, particularly cache usage. The system call latency results (Fig. 3a) show that LightRIM introduces an average overhead of 8.7% compared to enabling IMA alone. The highest impact is observed for open and close operations, with a maximum additional latency of 0.113 ms due to LightRIM's measurement process, which involves file operations. Despite this, the overall overhead is minimal and acceptable for practical deployment. The context switch latency results (Fig. 3b) indicate an average overhead of 2.3%, with the largest increase of 0.91 ms occurring under single-process configurations. This impact decreases as the number of parallel processes grows, confirming that LightRIM imposes negligible performance overhead, making it suitable for resource-constrained environments.

2) SPEC CPU2006: To evaluate the impact of LightRIM on compute-intensive applications, we conducted integer (INT) and floating-point (FP) tests on the SPEC CPU 2006 benchmark suite. Normalized CPU overhead was examined, revealing minimal impact in most applications. Fig. 4 illustrates the test results for the INT suite, indicating a mere 0.49% performance decline when the measurement module was en-

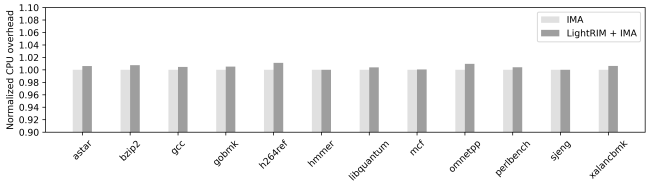


Fig. 4: Evaluation with SPEC CPU 2006 INT.

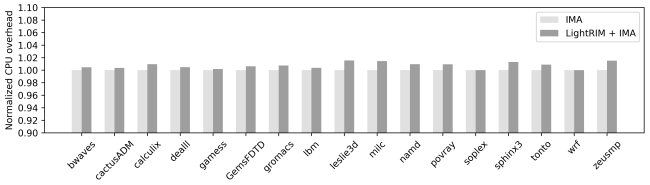


Fig. 5: Evaluation with SPEC CPU 2006 FP.

abled. Similarly, Fig. 5 presents the results for the FP suite, demonstrating only a 0.67% decrease in performance with the measurement module enabled.

F. Security Analysis

This subsection evaluates LightRIM's capability in detecting real-world attacks on immutable objects through two typical Linux integrity compromise scenarios.

1) Code Injection Attack: Attackers can compromise or control critical data by injecting malicious code, such as in VDSO hijacking attacks. These attacks involve hijacking the kernel's VDSO library to inject malicious code into processes. This allows attackers to access the process's memory, system, or network resources and potentially escalate privileges. Such attacks can evade standard security detection mechanisms as they operate within legitimate processes. Fortunately, we have discovered that monitoring the offset between VDSO functions and their starting addresses can effectively detect VDSO-based attacks. Any deviation from the baseline offset during measurement indicates potential tampering with the VDSO.

2) Rootkit Attack.: Rootkit technology manipulates kernel data structures to hide files, processes, network activity, and other critical system details. Diamorphine, a representative example, employs stealth syscall hooking by modifying the system call table, specifically `getdents` and `getdents64`, to conceal files and processes, and reuses the `kill` syscall for rootkit communication. Files with a specific prefix are hidden and processes receiving signal 31 are masked. The `diamorphine_init` module locates the syscall table address via brute force and integrates hooks, with `hacked_kill` enabling custom features. To counteract this threat, we established a baseline of syscall table addresses as a reference. By monitoring these values continuously at runtime, any unauthorized modifications to the syscall table can be promptly detected, providing effective protection against rootkit-based attacks.

V. Conclusion

LightRIM is a lightweight and adaptable integrity measurement framework tailored for resource-constrained embedded systems. This paper aims to protect unaltered critical objects during system runtime and promptly identify unauthorized modifications to maintain system security. Performance evaluations on LMBench and SPEC CPU2006 demonstrate its minimal overhead, highlighting its suitability for embedded applications. Future work will focus on deploying LightRIM in real-world embedded systems with known vulnerabilities to further assess its practicality and resilience.

References

[1] A. Apvrille, D. Gordon, S. E. Hallyn, M. Pourzandi, and V. Roy, "Digsig: Runtime authentication of binaries at kernel level." in *LISA*, vol. 4, 2004, pp. 59–66.

[2] C. Chang, X. Chen, S. Wang, Q. Xiao *et al.*, "Research on dynamic integrity measurement model based on memory paging mechanism," *Discrete Dynamics in Nature and Society*, vol. 2014, 2014.

[3] R. Sailer, X. Zhang, T. Jaeger, and L. Van Doorn, "Design and implementation of a tcg-based integrity measurement architecture." in *USENIX Security symposium*, vol. 13, no. 2004, 2004, pp. 223–238.

[4] B. Stelte, R. Koch, and M. Ullmann, "Towards integrity measurement in virtualized environments—a hypervisor based sensory integrity measurement architecture (sima)," in *2010 IEEE International Conference on Technologies for Homeland Security (HST)*. IEEE, 2010, pp. 106–112.

[5] T. Jaeger, R. Sailer, and U. Shankar, "Prima: policy-reduced integrity measurement architecture," in *Proceedings of the eleventh ACM symposium on Access control models and technologies*, 2006, pp. 19–28.

[6] W. Zhang, P. Liu, and T. Jaeger, "Analyzing the overhead of file protection by linux security modules," in *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, 2021, pp. 393–406.

[7] A. Rein, "Drive: Dynamic runtime integrity verification and evaluation," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, 2017, pp. 728–742.

[8] A. Litchfield and W. Du, "Xfilter: An extension of the integrity measurement architecture based on fine-grained policies," *Applied Sciences*, vol. 13, no. 10, p. 6046, 2023.

[9] F. Bohling, T. Mueller, M. Eckel, and J. Lindemann, "Subverting linux'integrity measurement architecture," in *Proceedings of the 15th International Conference on Availability, Reliability and Security*, 2020, pp. 1–10.

[10] Y. Qin, J. Liu, S. Zhao, D. Feng, and W. Feng, "Ripte: runtime integrity protection based on trusted execution for iot device," *Security and Communication Networks*, vol. 2020, pp. 1–14, 2020.

[11] T. Abera, N. Asokan, L. Davi, F. Koushanfar, A. Paverd, A.-R. Sadeghi, and G. Tsudik, "Things, trouble, trust: on building trust in iot systems," in *Proceedings of the 53rd Annual Design Automation Conference*, 2016, pp. 1–6.

[12] R. Rashmi and A. Karthikeyan, "Secure boot of embedded applications-a review," in *2018 Second International Conference on Electronics, Communication and Aerospace Technology (ICECA)*. IEEE, 2018, pp. 291–298.

[13] N. Sklavos, I. D. Zaharakis, A. Kameas, and A. Kalapodi, "Security & trusted devices in the context of internet of things (iot)," in *2017 Euromicro Conference on Digital System Design (DSD)*. IEEE, 2017, pp. 502–509.

[14] J. Gu, Y. Ma, B. Zheng, and C. Weng, "Outlier: Enabling effective measurement of hypervisor code integrity with group detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 3686–3698, 2021.

[15] W. Tan, Y. Chen, Y. Li, Y. Liu, J. Wu, Y. Ding, and C. Zhang, "Ptstore: Lightweight architectural support for page table isolation," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.

[16] J. Criswell, N. Dautenhahn, and V. Adve, "Kcofi: Complete control-flow integrity for commodity operating system kernels," in *2014 IEEE symposium on security and privacy*. IEEE, 2014, pp. 292–307.

[17] W. Tan, C. Li, Y. Chen, Y. Li, C. Zhang, and J. Wu, "Roload-pmp: Securing sensitive operations for kernels and bare-metal firmware," *IEEE Transactions on Computers*, 2024.